Sistema de Gestión de Torneos de Robótica Universitarios

---

1. Descripción General del Sistema

1.1 Nombre del sistema

Robotics Contest Management System (RCMS)

1.2 Objetivo

Desarrollar una aplicación web para la gestión integral de torneos de robótica universitarios que permita:

- Registro de alumnos
- Creación y administración de torneos
- Registro de equipos (máximo 2 integrantes)
- Gestión de modalidades/categorías
- Asignación de ganadores (1°, 2°, 3° lugar)
- Generación de certificados digitales
- Visualización de historial de participación

---

2. Alcance del Sistema

El sistema contempla dos tipos de usuarios:

2.1 Alumno

- Registro e inicio de sesión
- Creación de equipo
- Registro a torneos y modalidades
- Consulta de historial

- Consulta de resultados
- Descarga de certificados (si aplica)
- **Inicio de sesión con Google (OAuth 2.0)** *(solo alumnos)*

2.2 Administrador

- Crear, editar y cerrar torneos
- Crear y gestionar modalidades
- Visualizar equipos inscritos
- Eliminar o descalificar equipos
- Asignar 1°, 2° y 3° lugar por modalidad
- Generar certificados

---

3. Arquitectura General del Sistema

3.1 Arquitectura de Alto Nivel

Arquitectura desacoplada cliente-servidor:

Frontend (SPA híbrida SSR)
↓
API REST
↓
Base de Datos Relacional

---

4. Tecnologías Utilizadas

4.1 Frontend

- (App Router)
- React Server Components
- TypeScript
- TailwindCSS
- Middleware de Next.js

- Axios

4.2 Backend

- **Express.js**
- Arquitectura MVC con capa de servicios
- PostgreSQL
- JWT (autenticación)
- CORS
- Middleware personalizado

---

5. Arquitectura del Backend

5.1 Patrón Arquitectónico

Se implementa:

MVC + Service Layer + Repository Pattern

Separación de responsabilidades:

- Controllers → Manejo de HTTP
- Services → Lógica de negocio
- Repositories → Acceso a datos
- Prisma → ORM
- Middlewares → Autenticación y autorización

5.2 Estructura de Carpetas Backend

```
robotics-contests-api/
│
├── src/
│   ├── config/
│   ├── controllers/
│   ├── services/
│   ├── repositories/
```

```
│   ├── routes/
│   ├── middlewares/
│   ├── utils/
│   ├── prisma/
│   └── server.ts
│
├── prisma/
│   └── schema.prisma
│
├── package.json
└── .env
```

---

6. Arquitectura del Frontend

6.1 Patrón Arquitectónico

Feature-Based Modular Architecture + App Router

Separación por dominios de negocio.

6.2 Estructura de Carpetas Frontend

```
robotics-contests-frontend/
│
├── src/
│   ├── app/
│   │   ├── layout.tsx
│   │   ├── page.tsx
│   │   ├── login/
│   │   ├── contests/
│   │   ├── dashboard/
│   │   ├── admin/
│   │   └── loading.tsx
│   │
│   ├── features/
│   │   ├── auth/
│   │   ├── contests/
```

```
│   │   ├── teams/
│   │   └── admin/
│   │
│   ├── components/
│   ├── services/
│   ├── hooks/
│   ├── lib/
│   ├── types/
│   └── middleware.ts
│
├── package.json
└── .env.local
```

---

7. Modelo de Datos (Entidades)

7.1 User

Campo	Tipo	Descripción
id	UUID	Identificador
name	String	Nombre completo
email	String	Único
password	String	Hash
role	Enum	ADMIN / STUDENT
authProvider	Enum	LOCAL / GOOGLE
googleId	String | null	Sub/ID de Google (opcional)
createdAt	DateTime	Fecha creación

7.2 Contest

Campo	Tipo
id	UUID
title	String
description	String
date	DateTime

Campo	Tipo
location	String
status	Enum (OPEN, CLOSED, FINISHED)

7.3 Modality

Campo	Tipo
id	UUID
name	String
description	String
contestId	FK

Relación:
Contest 1 --- N Modality

7.4 Team

Campo	Tipo
id	UUID
name	String
contestId	FK
modalityId	FK
createdAt	DateTime

Restricción:

- Máximo 2 miembros por equipo (validación en Service)

7.5 TeamMember

Campo	Tipo
id	UUID
teamId	FK
userId	FK

Relación:
User N --- N Team (vía TeamMember)

7.6 Winner

Campo	Tipo
id	UUID
teamId	FK
modalityId	FK
position	Enum (1,2,3)

Restricción:

- Solo un ganador por posición por modalidad
- Máximo 3 registros por modalidad

---

8. Reglas de Negocio

1. Un equipo solo puede tener máximo 2 integrantes.
2. Un alumno puede participar en múltiples torneos.
3. Un equipo pertenece a una sola modalidad.
4. Solo pueden asignarse 3 ganadores por modalidad.
5. No se pueden registrar equipos si el torneo está CLOSED.
6. Solo ADMIN puede:
    - Crear torneos
    - Crear modalidades
    - Asignar ganadores
7. Los alumnos pueden autenticarse con Google.
8. Los administradores NO pueden usar Google Login (solo login tradicional con correo/contraseña).

---

9. Seguridad

9.1 Autenticación

- JWT firmado
- Token almacenado en httpOnly cookie
- Login con Google (OAuth 2.0) para alumnos
  - El backend valida el id_token de Google
  - Si el correo ya existe, inicia sesión
  - Si no existe, crea usuario con role STUDENT

9.2 Autorización

RBAC (Role-Based Access Control)

Middleware backend:

- authMiddleware
- roleMiddleware

Middleware frontend:

- Protección de rutas `/admin`
- Protección de `/dashboard`

---

10. Flujo General del Sistema

Registro de equipo

Alumno →
Crear equipo →
Seleccionar torneo abierto →
Seleccionar modalidad →
Confirmar inscripción

Asignación de ganadores

Admin →
Seleccionar modalidad →
Asignar posición (1,2,3) →
Generar certificado

---

11. Generación de Certificados

Se generarán desde backend:

- Generación dinámica en PDF
- Contendrá:
  - Nombre alumno
  - Nombre equipo
  - Modalidad
  - Posición
  - Fecha
  - Firma institucional

---

12. API REST Endpoints Principales

Auth

POST /auth/login

POST /auth/register

POST /auth/google (STUDENT)

Contests

GET /contests

POST /contests (ADMIN)

PATCH /contests/:id

DELETE /contests/:id

Teams

POST /teams

GET /teams/my

Winners

POST /winners (ADMIN)

GET /winners/modality/:id

---

13. Principios Aplicados

- Separation of Concerns
- Single Responsibility Principle
- Clean Architecture ligera
- Modularización por dominio
- Control de acceso basado en roles
- Integridad referencial en base de datos
- Validaciones en frontend y backend

---

14. Decisiones Arquitectónicas Justificadas

¿Por qué Next.js?

- SSR y Server Components
- Middleware nativo
- Layouts anidados
- Loading automático

¿Por qué Express?

- Flexibilidad
- Arquitectura MVC clara
- Middleware personalizable

¿Por qué Prisma?

- Tipado fuerte
- Migraciones controladas
- Relaciones explícitas

---

15. Extensiones Futuras

- Multi-universidad (modo SaaS)
- Panel estadístico
- Rankings históricos
- Exportación a Excel
- Firma digital avanzada

Documento de API REST